

Análise Detalhada: Plataforma de Partilha de Boleias em Microsserviços

Autores: Dercio Sinione Domingos (20220103), Joel Dialamicua (20221985)

Data de Referência: 2026-02-02



Visão Geral e Proposta de Valor

Este documento detalha a arquitetura e implementação de uma plataforma de partilha de boleias (carpooling), concebida como um sistema de Aplicações Distribuídas. O projeto, desenvolvido por Dercio Domingos e Joel Dialamicua, adota uma abordagem moderna baseada em microsserviços, orquestrada com Docker Swarm e com autenticação via JWT (JSON Web Tokens). A solução visa oferecer um ecossistema completo e resiliente para condutores e passageiros, cobrindo todo o ciclo de vida de uma viagem partilhada.

O sistema foi projetado para ser modular, escalável e de fácil manutenção, separando as responsabilidades de negócio em serviços independentes. As funcionalidades centrais incluem:

- **Gestão de Identidade e Perfis:** Registo, autenticação segura e gestão de papéis distintos (Condutor, Passageiro, Administrador).
- **Ciclo de Vida das Viagens:** Publicação, pesquisa avançada, sistema de reservas com aprovação e cancelamento.
- **Transações e Custos:** Simulação de pagamentos e gestão de despesas associadas a cada viagem.
- **Inteligência de Localização:** Integração com serviços de GPS para cálculo de rotas, distâncias e geolocalização.
- **Comunicação Assíncrona:** Sistema de notificações para manter os utilizadores informados sobre eventos relevantes.
- **Administração e Monitorização:** Um painel de controlo centralizado para gestão global e observabilidade do sistema.



Arquitetura de Microsserviços: O Coração do Sistema

A espinha dorsal da plataforma é a sua arquitetura de microsserviços. Cada serviço é autônomo, focado numa capacidade de negócio específica, e comunica com os outros através de APIs bem definidas. Esta abordagem promove a resiliência, escalabilidade individual dos componentes e agilidade no desenvolvimento.

Ponto de Entrada Único: Todas as requisições externas são direcionadas para o Cloud Gateway no endereço `http://localhost:8888`, que atua como um proxy reverso, encaminhando o tráfego para o serviço apropriado com base em prefixos de rota (ex: `/identity`, `/trips`).

1) Service Discovery (Eureka)

Num ambiente distribuído dinâmico, os serviços precisam de se encontrar uns aos outros. O Eureka atua como um Service Registry, onde cada microsserviço se registra automaticamente ao iniciar. Isto elimina a necessidade de configurações de rede estáticas, permitindo que os serviços descubram dinamicamente as localizações (endereço IP e porta) dos outros. Esta capacidade é fundamental para o balanceamento de carga e a resiliência do sistema, pois o Gateway pode consultar o Eureka para encontrar instâncias saudáveis de um serviço antes de encaminhar uma requisição.

2) Cloud Gateway

O Cloud Gateway é a porta de entrada para todo o ecossistema de backend. Ele desempenha várias funções críticas:

- Ponto Único de Entrada (Single Point of Entry): Simplifica a arquitetura para o cliente (frontend), que só precisa de conhecer um único endereço.
- Roteamento Dinâmico: Utiliza a informação do Eureka para encaminhar as requisições para as instâncias corretas dos microsserviços. Por exemplo, uma chamada para `/identity/api/v1/auth/login` é automaticamente direcionada para o Identity Service.

- Centralização de Lógicas Transversais (Cross-Cutting Concerns): É o local ideal para implementar segurança (validação de tokens JWT), rate limiting, logging de requisições e políticas de CORS, evitando a duplicação deste código em cada serviço.

3) Identity Service

Este é um dos serviços mais críticos, responsável pela segurança e gestão de utilizadores. As suas responsabilidades incluem a autenticação (login com email/senha e geração de JWTs), o registo de novos utilizadores e a gestão completa de perfis. Ele armazena informações sobre os papéis (Condutor, Passageiro, Admin), controlando o que cada utilizador pode fazer no sistema. Além disso, gere o sistema de avaliações (ratings) entre utilizadores, um pilar para construir confiança na comunidade. Fornece também endpoints administrativos para tarefas como a criação de novos administradores ou o bloqueio de contas.

4) Vehicles Service

Totalmente desacoplado da lógica de utilizadores, este serviço foca-se exclusivamente na gestão de veículos. Permite que os condutores registem os seus carros, associando-os ao seu perfil de condutor através de um ID. O serviço mantém um catálogo de marcas e modelos, garantindo a consistência dos dados. Esta separação de responsabilidades é um exemplo claro dos benefícios dos microsserviços: a lógica de veículos pode evoluir independentemente da lógica de utilizadores.

5) Trips Service

Este serviço orquestra a principal lógica de negócio da plataforma. Ele gere todo o ciclo de vida de uma viagem, desde a sua criação por um condutor até à sua conclusão. As suas funções incluem a publicação de viagens (com detalhes como origem, destino, data, vagas), a pesquisa por parte dos passageiros e a gestão de reservas (pedidos, aceitações, rejeições e cancelamentos). É também responsável por calcular o custo por pessoa e gerir o estado da viagem (ex: **aberta**, **iniciada**, **concluída**), interagindo com outros serviços quando necessário (ex: com o GPS Service para calcular distâncias).

6) Payments Service

Para manter a lógica de pagamentos isolada, este serviço simula as transações financeiras. Embora não processe pagamentos reais, ele regista cada transação simulada associada a uma viagem. Isto permite consultar o histórico de pagamentos, gerar relatórios financeiros e, futuramente, integrar um gateway de pagamento real sem impactar os outros serviços.

7) GPS Service

Este serviço abstrai todas as funcionalidades relacionadas com geolocalização. Ele oferece endpoints para pesquisar locais, realizar reverse geocoding (converter coordenadas em moradas), e, crucialmente, calcular a distância entre dois pontos. Esta última função é essencial para o Trips Service estimar custos e tempos de viagem. Também pode gerir localizações recentes de um utilizador para melhorar a experiência de pesquisa.

8) Notifications Service

A comunicação eficaz com o utilizador é vital. Este serviço gere a criação e entrega de notificações em tempo real ou assíncronas. Exemplos incluem: "A sua reserva foi aceite", "A viagem está prestes a começar", "Recebeu uma nova avaliação". O serviço garante o isolamento de dados, assegurando que cada utilizador apenas vê as suas próprias notificações, e permite a gestão do estado das mesmas (lida/não lida, eliminação).

Frontend (Web): A Interface com o Utilizador

A interface web foi desenvolvida com Spring Boot + Thymeleaf, uma escolha que permite uma forte integração com o ecossistema backend, também baseado em Spring. A aplicação é renderizada no lado do servidor e oferece uma experiência rica e segmentada para os diferentes tipos de utilizador.

- **Área Pública:** Acessível a todos, permite pesquisar viagens e visualizar detalhes sem necessidade de autenticação, promovendo a descoberta da plataforma.
- **Dashboards de Utilizador:** Após o login, os utilizadores são direcionados para dashboards personalizados. O Passageiro pode gerir as suas reservas e pagamentos, enquanto o Condutor pode gerir as suas viagens publicadas, veículos e pedidos de reserva.

- Publicação de Viagem: Um fluxo guiado em várias etapas para facilitar a criação de uma nova oferta de boleia.
- Painel Administrativo: Uma área restrita e poderosa que oferece uma visão global do sistema. Está dividida em secções para gestão de utilizadores, viagens, veículos, pagamentos, e para a visualização de relatórios e métricas de monitorização, permitindo um controlo total sobre a plataforma.



Orquestração e Implantação com Docker Swarm

A complexidade de gerir múltiplos serviços é resolvida através da orquestração de contentores com Docker Swarm. Esta ferramenta, nativa do Docker, permite definir e executar toda a aplicação multi-serviço de forma declarativa.

O ficheiro `backend/docker-stack.yml` é o manifesto que descreve todos os componentes da stack: os microsserviços, a aplicação frontend, as bases de dados e a stack de monitorização. O Docker Swarm garante que os serviços são implantados, reiniciados em caso de falha e podem comunicar entre si através de uma rede overlay partilhada. A automação é garantida pelo script `backend/deploy_swarm.sh`, que realiza o build das imagens Docker e faz o deploy de toda a stack, simplificando drasticamente o processo de implantação.



Monitorização e Observabilidade: A Visão Interna

Um sistema distribuído robusto exige uma forte capacidade de observabilidade. A plataforma integra uma stack de monitorização completa, acessível localmente, que fornece insights profundos sobre a saúde e o desempenho do sistema.

- Spring Boot Actuator: Cada microsserviço expõe endpoints (ex: `/health`, `/metrics`) que fornecem informações vitais sobre o seu estado.
- Prometheus (`http://localhost:9090`): Uma ferramenta de monitorização que periodicamente "raspa" (scrapes) as métricas expostas pelo Actuator, armazenando-as como séries temporais.

- Grafana (<http://localhost:3000>): Uma plataforma de visualização que se conecta ao Prometheus para criar dashboards interativos. Estes painéis permitem monitorizar em tempo real a latência das requisições, taxas de erro, uso de CPU/memória, e outros indicadores chave de desempenho (KPIs).
- Loki + Promtail: Esta combinação oferece um sistema de agregação de logs centralizado. O Promtail recolhe os logs de todos os contentores e envia-os para o Loki, que os indexa. Isto permite pesquisar e analisar logs de toda a aplicação a partir de um único local (geralmente através do Grafana), o que é indispensável para a depuração de problemas em ambientes distribuídos.

A documentação completa de todos os endpoints, incluindo os de API (Swagger) e os do Actuator, está consolidada no ficheiro `Endpoints.md`, servindo como um guia de referência essencial para desenvolvedores e administradores.

Conclusão

O projeto da plataforma de partilha de boleias demonstra com sucesso a aplicação de princípios modernos de engenharia de software para construir um sistema distribuído complexo. A escolha de uma arquitetura de microsserviços, aliada à orquestração com Docker Swarm e a uma stack de observabilidade completa, resulta numa solução que é simultaneamente robusta, escalável e manutenível.

A clara separação de responsabilidades entre os serviços, a segurança centralizada no Gateway com JWT, e a experiência de utilizador segmentada no frontend, compõem um ecossistema coeso e funcional. Este trabalho não só resolve um problema de negócio concreto, mas também serve como um excelente caso de estudo sobre como projetar, implantar e monitorizar aplicações distribuídas no mundo real.